

Job Applications Management System

Laravel + AJAX – Full Implementation Guide

This document provides a complete, clean, and production-ready guide for implementing an Admin Job Applications management module using Laravel, AJAX, Bootstrap, and jQuery. It includes database setup, model configuration, controller logic, routes, Blade UI, AJAX interactions, and common error fixes.

1. Database Structure

Table Name: job_applications

This table stores all job application submissions from candidates.

Columns:

- id (Primary Key)
- job_requisition_id (Foreign Key reference)
- full_name
- email
- phone
- resume (stored file path)
- cover_letter
- status (pending / reviewed / shortlisted / rejected)
- created_at, updated_at

2. Migration

Create a migration to generate the `job_applications` table. Ensure the table name matches exactly with the Eloquent model.

Key Notes:

- Table name must be plural (`job_applications`).
- Status column defaults to 'pending'.
- Resume file is stored under `storage/app/public`.

3. Eloquent Model Configuration

Model: JobApplication

The model explicitly defines the table name to avoid environment-related issues.

Important Configurations:

- `protected $table = 'job_applications';`
- `$fillable` array for mass assignment protection
- Accessor for resume download URL

4. Controller Responsibilities

Controller: Admin\JobApplicationController

Main Methods:

- index() – Load all applications
- updateStatus() – AJAX-based status update
- show() – Return application details as JSON
- downloadResume() – Secure resume download

5. Routes Definition

All routes are grouped under the admin prefix for security and clarity.

Endpoints:

- GET /admin/job-applications
- POST /admin/job-applications/status
- GET /admin/job-applications/{id}
- GET /admin/job-applications/{id}/download

6. Blade UI Overview

The Blade view provides a clean admin interface built with Bootstrap.

Features:

- Responsive table layout
- Inline status dropdown
- Action buttons for viewing and downloading resumes
- Bootstrap modal for detailed application view

7. AJAX Functionality

AJAX is implemented using jQuery to avoid page reloads.

AJAX Actions:

- Status change sends POST request
- View button fetches application data dynamically
- Modal is populated using returned JSON

8. Resume Download Handling

Resumes are securely downloaded using Laravel's `Response::download` method.

Storage Requirements:

- Files stored in `storage/app/public`
- `php artisan storage:link` must be executed

9. Common Errors & Fixes

Error: Table not found (42S02)

Fixes:

- Ensure correct database selected in .env
- Confirm migration has run
- Explicitly define table name in model
- Clear config cache after changes

10. Final Result

This implementation provides a professional-grade admin experience with:

- Zero page reloads
- Clean and maintainable codebase
- Secure resume handling
- Scalable structure for future enhancements